

SmartCourier AI

La copia web, probada.

La aplicación está disponible localmente en **http://127.0.0.1:5173**. La copia usa su propia base smartcourier_web, API en 5002 y servicio Python en 8001. El código móvil se conserva en frontend-mobile/.

Qué quedó funcionando

Cuenta y sesión; comparador con pausa, reanudación y velocidades; seis eventos; mapa Leaflet; recuperación de rutas al recargar en pausa; historial filtrado; ajustes; y laboratorio **Agente Courier** para consultar el modelo entrenado con variables editables.

Pruebas terminadas

Verificación	Resultado
Backend con PostgreSQL y Python reales	76 pruebas aprobadas
Navegador Chromium	2 recorridos completos aprobados; escritorio y móvil
Compilación de producción	Vite genera frontend/dist/
Regresión económica	Pago sin propina, costo marginal, plazos e inputs inválidos
Recuperación	Snapshot, replay y reinicio sin cobros duplicados

Resultados en escenarios independientes

Módulo	Ganancia positiva	En cero	Con pérdida
Comparador web	162 / 162	0	0
Courier entrenado	106 / 108	2	0

Son resultados de simulación. No prueban ganancias garantizadas en condiciones reales ni que SmartCourier gane a otra estrategia en todos los turnos. Los dos módulos tienen modelos de costo diferentes.

Seguridad, costo y valor del tiempo.

El tier 1 decide ACCEPT o SKIP sin llamar al modelo dentro de la ventana de decisión. El tier 2 en Python calcula el salario de reserva con parámetros entrenados. La consulta web prepara una estrategia independiente por solicitud; no altera el turno global de los jueces.

Secuencia aplicada en el código

1. Validar datos. Distancias, pagos, peso, volumen y tiempos numéricos deben ser finitos y no negativos; surge debe ser mayor que cero.
2. Planear el viaje: ir a recoger, esperar preparación y entregar. Si hay un pedido en curso, comparar terminarlo primero con recoger el nuevo primero.
3. Aplicar seguridad: zonas nocturnas, descanso, calor, fin del turno y capacidad. El pago no puede revocar estos rechazos.
4. Comprobar pedidos simultáneos y plazos; máximo dos pedidos, ruta factible.
5. Exigir que el pago base con surge supere el combustible, a centavos, sin depender de propina.
6. Comparar tarifa ajustada con salario de reserva. Un empate a centavos se acepta.

Fórmulas económicas

Bruto estimado = pago base × surge + propina estimada

Combustible = kilómetros marginales × costo por km

Neto estimado = bruto estimado - combustible

Neto sin propina = pago base × surge - combustible; debe ser > 0

Tarifa ajustada/h = $60 \times (\text{neto} - \text{costo del próximo traslado vacío}) / (\text{minutos marginales} + \text{espera estimada en destino})$. El tiempo marginal se limita como mínimo a un minuto.

Límites conservados

Zonas 3 y 10: no entregar de 22:00 a 06:00. Descanso de 20 min tras 240 min continuos. Ventana de calor 12:00-16:00: máximo 90 min continuos. Fin del turno leído del estado. Margen de seguridad: **5 min**.

Vehículo	Combustible/km	Peso / volumen	Velocidad base
Bici	\$0.00	7 kg / 25 L	14 km/h
Moto	\$1.20	12 kg / 45 L	28 km/h
Auto	\$2.40	40 kg / 200 L	22 km/h

Fuentes del código: backend/src/courier/{fastPath,trip,safety,config,demandModel}.js y optimization-service/main.py. La bicicleta tiene combustible cero en este protocolo; no significa que trabajar en bici no tenga otros costos.

Medir sin ajustar con las respuestas del examen.

courier/ aporta protocolo, esquemas, ejemplos y validadores; no es un historial masivo de entregas reales. Los flujos son sintéticos reproducibles. Demanda y reservas se entrenaron con semillas **1-54**. Se probaron reservas $\times 1$, $\times 0.75$, $\times 0.5$, $\times 0$ y $\times 1.25$ por vehículo, y factores de fin de turno 0.5, 0.25 y 0. Los parámetros originales siguieron siendo los mejores de esos candidatos y se conservaron.

Reserva aprendida, MXN/h antes del ajuste de fin

Vehículo	Mañana	Comida	Tarde	Cena	Noche
moto	80	0	90	120	70
car	80	80	40	90	30
bike	80	0	60	0	0

Se reduce a la mitad durante los últimos 45 min. Factores de lluvia y surge de la estrategia: 1. El hash/versionado identifica el artefacto: courier-web-8b72b5f38fa4.

Courier: 108 semillas nuevas, 130001-130108

Política con restricciones	Neto medio/turno
SmartCourier	\$501.82
AcceptAll + Safety	\$477.74
GreedyRate + Safety	\$492.06

804 pedidos aceptados, cero aceptaciones con neto sin propina no positivo, **cero violaciones de seguridad**, 2 entregas tardías y cero diferencias en el replay de 108 turnos. Neto medio eliminando todas las propinas de los pedidos seleccionados: **\$418.44**. P99 máximo del cálculo rápido: 0.14 ms; presupuesto 50 ms, sin incluir HTTP ni refresco de estrategia.

Dónde aún puede perder frente a otra política

SmartCourier no gana todos los enfrentamientos: frente a AcceptAll + Safety registra 45 victorias, 27 empates y 36 derrotas por ingreso neto del turno. En auto, GreedyRate + Safety tiene mayor media; en bici, AcceptAll + Safety. Dos turnos de auto terminan con ingreso cero. No se reajustaron parámetros con estas semillas.

Las referencias +Safety pasan por las mismas restricciones de ruta y piso de neto sin propina. El reporte oficial adicional de 30 semillas conserva OurAgent \$466.17 frente a \$458.36 y \$448.39; AcceptAll sin reglas logra \$543.71 con 87 violaciones.

Una web operativa. Resultados trazables.

El comparador usa puntajes, planificación anticipada y agrupación de pedidos; conserva la física compartida entre Baseline y SmartCourier. Costos estimados por km: bici \$0.40, moto \$2.00 y auto \$4.50, incluyendo combustible, mantenimiento y depreciación. Estos resultados no se mezclan con Courier.

162 pruebas: 54 semillas nuevas × 3 escenarios

Vehículo (54 pruebas)	Neto Smart	Neto Baseline	Victorias/h
motorcycle	\$551.91	\$379.86	50/54
bike	\$374.36	\$266.03	50/54
car	\$337.11	\$113.09	52/54

Media total de SmartCourier: **\$421.13** por turno frente a \$252.99. Todas las 162 pruebas terminan con ganancia; mínimo \$45.35. Supera a Baseline por neto/hora trabajada en 152/162. Escenarios: normal, tráfico severo y cierre vial, con rutas estimadas deterministas.

No es una garantía universal

El comparador registró 189 entregas tardías frente a 190 de Baseline. En moto y bici tuvo más retrasos que Baseline, aunque mayores ingresos. Courier no incluye desgaste, impuestos ni gastos personales; los resultados sin propina son una sensibilidad contable de los pedidos seleccionados, no un reentrenamiento. Un cambio futuro de pago, costo, cancelación o ruta puede cambiar la rentabilidad real.

Arranque local y archivos de referencia

Desde smartcourier-web/: **npm start** inicia web, API y Python en segundo plano; **npm run status** verifica los tres servicios; **npm stop** los detiene. Registra una cuenta nueva. El Mac debe permanecer encendido. README_WEB.md contiene instalación, pruebas y límites de despliegue.

Pruebas: npm run test:backend; npm run test:web; npm run build.

Evaluación: node backend/scripts/courier/refineWeb.js holdout; node backend/scripts/validateComparison.js. Los resultados por semilla y decisiones están en output/validation/. El estado inicial se conserva en COPY_MANIFEST.json y las nueve modificaciones quedan registradas en COPY_AMENDMENTS.json.

Este informe actualiza la documentación original de la copia. El sistema queda listo para demostración local; un despliegue público requiere hosting, HTTPS, base propia y aislamiento de los endpoints de jueces.